郑州轻工业大学

本科毕业设计（论文）

题　　目　　校园物资智能管理系统设计与实现

学生姓名　　刘祎凯

专业班级　　软件工程 22-06 班

学　　号　　542213340621

学　　院　　软件学院

指导教师（职称）　　周开来

完成时间　　2026 年 5 月 18 日

郑州轻工业大学

毕业设计（论文）任务书

题目　　校园物资智能管理系统设计与实现

专业　　软件工程　　学号　　542213340621　　姓名　　刘祎凯

主要内容：本课题围绕高校物资管理中台账分散、库存批次难追踪、申领审批链条不清晰和预警处置滞后等问题，设计并实现校园物资智能管理系统。系统采用前后端分离架构，后端基于 Spring Boot、Spring Security、JWT、MyBatis-Plus 和 MySQL，前端基于 Vue 3、Vite、Pinia、Element Plus 与 ECharts，完成基础数据、库存批次、申领审批、调拨协同、预警处理、通知日志和统计分析等功能。

基本要求：系统功能应与真实代码、数据库脚本和测试材料一致；论文应围绕需求分析、总体设计、数据库设计、详细实现和系统测试展开；智能化表述限于库存预警、效期规则、候选仓排序、补货建议和统计分析等可复核内容。

思政要求：课题应体现服务校园治理、规范物资流转、强化数据安全和审计追踪的责任意识，在系统设计与论文写作中坚持真实、严谨和可验证原则。

主要参考资料：

[1] 李晓英，高菲. 智慧校园背景下的高职院校物资管理系统建设创新研究[J]. 现代物业(中旬刊)，2019(09)：12.

[4] 张俊萌，张华贵. 医疗机构物资流通管理信息系统的设计与实现[J]. 工业控制计算机，2026，39(04)：132-134.

[6] 崔靖茹，文华，刘宏磊，等. 基于Vue和SpringBoot框架的高校信息化项目管理系统的设计与实现[J]. 现代信息科技，2025，9(22)：77-81.

[10] Kevin Santiago Rey Rodriguez, Julián David Avellaneda Galindo, Josep Tárrega Juan, et al. Secure Development Methodology for Full Stack Web Applications: Proof of the Methodology Applied to Vue.js, Spring Boot and MySQL[J]. Computers, Materials & Continua, 2025, 85(1).

[17] 张文亮. MySQL 应用实战与性能调优[M]. 北京：机械工业出版社，2022.

完　成　期　限：2026 年 5 月 18 日

指导教师签名：

专业负责人签名：

年　　月　　日

校园物资智能管理系统设计与实现

摘 要：针对高校物资管理中台账分散、审批链路不清晰、库存批次难追踪和预警处理滞后等问题，本文设计并实现了一套校园物资智能管理系统。系统采用前后端分离架构，前端以 Vue 3、Vite、Pinia、Element Plus 和 ECharts 组织业务页面与统计展示，后端基于 Spring Boot、Spring Security、JWT、MyBatis-Plus 和 MySQL 实现认证授权、基础数据管理、库存批次管理、申领审批、仓间调拨、预警处理、通知日志和统计分析等功能。论文围绕真实项目，重点说明需求分析、系统设计、数据库结构、关键实现与测试验证过程。经后端自动化测试、前端构建验证、单元测试和端到端流程验证，系统主要功能能够稳定运行并达到预期目标。该系统可为高校物资流转、库存追踪和风险提醒提供较为规范的信息化支撑。

关键词：校园物资管理；前后端分离；库存批次；申领审批；库存预警

Design and Implementation of Campus Material Intelligent Management System

Abstract: To address scattered ledgers, unclear approval chains, weak batch traceability, and delayed warning handling in campus material management, this thesis designs and implements a campus material intelligent management system. The system adopts a front-end/back-end separated architecture. The front end uses Vue 3, Vite, Pinia, Element Plus, and ECharts to organize business pages and statistical views, while the back end uses Spring Boot, Spring Security, JWT, MyBatis-Plus, and MySQL to implement authentication, master data management, inventory batch management, requisition approval, warehouse transfer, warning handling, notification logging, and statistical analysis. The thesis focuses on requirement analysis, system design, database structure, key implementation details, and test verification based on the real project repository. Results from back-end automated tests, front-end build verification, unit tests, and end-to-end tests show that the main functions run stably and meet the expected goal. The system can provide practical information support for campus material circulation, inventory tracking, and risk warning.

Keywords: campus material management; front-end/back-end separation; batch inventory; requisition approval; inventory warning

目 录

1 绪论 1
1.1 研究背景和意义 1
1.2 国内外研究现状与分析 2
1.2.1 国内研究现状 2
1.2.2 国外相关实践启示 2
1.3 研究内容 3
1.4 本章小结 3
2 相关技术 4
2.1 技术选型概述 4
2.2 前端开发技术 4
2.3 后端与数据访问技术 4
2.4 安全认证与可视化技术 4
2.5 本章小结 5
3 系统需求分析 6
3.1 可行性分析 6
3.1.1 技术可行性 6
3.1.2 经济可行性 6
3.1.3 社会可行性 6
3.1.4 法律可行性 6
3.2 功能性需求分析 6
3.2.1 系统结构分析 6
3.2.2 基础数据管理需求 7
3.2.3 库存批次管理需求 7
3.2.4 申领审批需求 7

3.2.5 调拨协同需求 7
3.2.6 预警统计需求 8
3.2.7 关键业务流程与用例说明 9
3.3 非功能性需求分析 11
3.4 本章小结 12
4 系统设计 13
4.1 系统总体架构设计 13
4.2 系统模块设计 14
4.2.1 接口与安全设计 14
4.2.2 智能能力设计 15
4.3 数据库设计 16
4.3.1 数据库设计原则 16
4.3.2 实体关系设计 16
4.3.3 关键数据表设计 17
4.3.4 索引与约束设计 18
4.4 本章小结 19
5 系统实现 20
5.1 开发环境搭建 20
5.2 功能实现 20
5.2.1 认证授权与登录态管理实现 20
5.2.2 基础数据管理实现 22
5.2.3 库存批次与出入库实现 22
5.2.4 申领审批与调拨实现 23
5.2.5 预警、补货建议与统计分析实现 26
5.2.6 日志通知与事件管理实现 28
5.3 本章小结 29

6 系统测试 30
6.1 系统安装部署 30
6.2 软件功能测试 30
6.2.1 后端自动化测试 30
6.2.2 前端构建、单测与 E2E 验证 30
6.2.3 业务场景测试 31
6.2.4 性能基线与边界说明 31
6.3 本章小结 32
结束语 33
致　谢 34
参考文献 35
附录 1 关键接口与测试补充说明 37

1 绪论

1.1 研究背景和意义

高校日常运行离不开实验耗材、办公用品、医疗防护、清洁消杀、应急照明和设备备件等物资。传统管理方式常依靠纸质台账、分散 Excel 表和人工沟通完成，数据更新滞后、审批记录不完整、库存批次不透明等问题会直接影响物资调配效率。智慧校园建设要求后勤保障业务逐步从单点记录转向流程协同和数据治理，物资管理系统因此需要承担主数据维护、库存追踪、审批流转和风险提醒等职责[1][2]。

从物资管理研究和行业实践看，信息化系统通常将入库、领用、审核、库存监控等环节统一管理，以减少人工统计和重复沟通成本[3][4]。高校实验室和后勤物资还涉及有效期、批次和安全合规问题，相关研究强调全生命周期管理、身份认证、异常提醒和后续数据分析的重要性[5]。本课题正是在这一背景下，将校园物资从入库、申领、审批、出库、调拨到预警处置的核心链路转化为可追踪的数据流程。

本文所述“智能管理”主要体现为规则化判断、候选仓排序、批次优先扣减、补货建议和图表统计等辅助能力。这些功能具有规则可解释、结果可复核的特点，不涉及复杂人工智能模型。

本课题的应用意义在于将校园物资管理中原本分散的角色、单据、库存和日志整理为统一流程。部门用户提交申领后，审批人员可以基于单据状态进行处理，仓库管理员再根据库存批次执行出库或调拨，最终形成可回溯的业务闭环。这种处理方式能够减少口头确认和线下台账之间的信息偏差。

本课题的工程重点在于将需求分析、数据库设计、前后端分离、权限控制、事务处理和测试验证结合起来，形成面向校园物资管理的完整业务系统。系统围绕 sys_user、material_info、inventory_batch、apply_order、transfer_order 和 warning_record 等真实数据对象组织功能，需求、设计、实现和测试之间具有明确对应关系。

从安全和管理角度看，系统通过 JWT 登录认证、角色菜单、接口权限和日志记录形成基本的访问边界。相关研究表明，前后端分离项目中的 JWT 与 RBAC 能够降低会话依赖并提升权限管理清晰度[19]，但也需要关注业务逻辑漏洞和令牌使用风险[20]。本系统在实现中采用刷新令牌持久化、登录日志和操作日志等机制，为后续管理和排查提供依据。

1.2 国内外研究现状与分析

1.2.1 国内研究现状

国内有关高校信息化和管理系统的研究较多，基于 Spring Boot 与 Vue 的前后端分离方案已经广泛用于高校项目管理、事故报告数据库、研究生档案管理和校园服务平台等场景[6][7][8][9]。这些系统通常采用 B/S 架构，将用户管理、角色权限、业务单据、数据查询和统计展示拆分为若干模块，适合本科毕业设计中说明需求分析、总体设计、数据库设计和系统测试。

与本课题更接近的研究集中在物资流通、后勤物资、实验室危化品和库存监控等方向。医疗机构物资流通系统强调入库、借用、领用、审核和库存监控的连续处理[4]，高校危化品溯源研究强调从采购、入库、使用到报废的全生命周期闭环[5]。这些研究为校园物资系统提供了流程建模参考，但本文不直接照搬其业务场景，而是以当前项目中已有代码和数据库为准。

国内近年的智能管理研究也开始关注预警、流程自动化和数据分析。例如入厂煤智能管理平台将流程管控和预警模型结合起来[21]，智慧实验室管理研究讨论了更复杂的人工智能方法[22]。与这些研究相比，本系统的实现范围更聚焦于本科毕业设计场景，主要采用固定阈值、时间窗口、移动平均和候选排序等轻量方法。

1.2.2 国外相关实践启示

国外文献中与本课题直接相关的主要是 Web 应用系统工程实践和全栈安全方法。有关 Vue.js、Spring Boot 和 MySQL 的全栈安全方法研究强调身份认证、授权控制、错误处理、日志和输入校验等安全维度[10]。Spring Boot 和 Vue 也被用于合同管理、学生考勤和工时管理等信息系统实现[11][12][13]，说明该技术路线适合构建以业务流程为中心的管理系统。

这些实践的启示在于，管理系统的价值不仅来自页面数量，也来自业务流程是否稳定、数据模型是否清楚、权限边界是否明确、测试是否可复核。本文后续章节因此不追求复杂理论模型，而是围绕本项目的真实功能说明系统如何支持校园物资的申领、审批、库存、调拨和预警。

1.3 研究内容

本文围绕校园物资智能管理系统的实际建设过程展开研究，主要内容包括：分析校园物资管理场景中的角色分工、业务链路和功能需求；结合前后端分离架构完成系统总体设计、模块设计与数据库设计；围绕认证授权、库存批次、申领审批、调拨协同、预警处理、通知日志和统计分析说明关键实现过程；最后结合自动化测试、前端构建验证和典型业务场景验证，说明系统实现效果与适用边界。全文以真实代码、SQL 脚本、页面功能和测试材料为依据，不引入仓库中不存在的模块、接口和指标。

1.4 本章小结

本章从校园物资管理的信息化背景出发，说明了课题的研究意义，梳理了相关研究现状，并明确了本文的研究范围与写作主线。后续章节将在此基础上，对系统采用的关键技术、需求分析、系统设计、功能实现与测试验证进行展开。

2 相关技术

2.1 技术选型概述

本系统采用前后端分离技术路线。前端负责页面展示、表单交互、路由跳转和图表呈现，后端负责身份认证、权限控制、业务规则、事务处理和数据访问。此类技术组合在高校管理系统和通用信息系统中较为常见，能够较好支撑角色隔离、列表查询和流程审批。根据 frontend/package.json 和 backend/pom.xml，系统实际使用的核心技术包括 Vue 3、Vite、Pinia、Element Plus、Axios、ECharts、Spring Boot 3.3.5、Spring Security、JWT、MyBatis-Plus、JdbcTemplate、MySQL 8 和 H2。论文只介绍这些在项目中承担实际功能的技术，避免脱离系统实现去罗列与课题无关的框架。

2.2 前端开发技术

Vue 3 用于构建前端页面，配合 Vue Router 管理登录页、仪表盘、基础数据、库存、申领、调拨、预警、统计分析和系统工具等路由。Pinia 用于保存登录状态和用户信息，Axios 封装后端接口请求，Element Plus 提供表单、表格、对话框、按钮和分页等基础组件。

Vite 用于前端工程构建，能够提供较快的本地开发启动和生产构建能力，适合本系统这类多页面管理端。ECharts 用于库存占比、出入库趋势、仓库分布、部门排行和效期统计等图表展示，相关功能主要承担统计分析和可视化展示任务，不涉及脱离业务场景的大数据平台。

2.3 后端与数据访问技术

后端基于 Spring Boot 3.3.5 构建 REST API，使用 Controller、Service、Mapper 和 Entity 分层组织代码。Spring Boot 能够简化配置管理、依赖装配和测试集成，在管理信息系统开发中具有较好的工程适配性。MyBatis-Plus 负责实体映射和通用 CRUD，JdbcTemplate 用于统计分析和补货建议中的聚合查询。

数据层使用 MySQL 8 作为业务数据库，测试和截图场景使用 H2 模拟 MySQL 模式。MySQL 在中小型管理系统中具有成熟的事务、索引和查询能力，本系统的索引主要覆盖用户角色、批次出库、预警查询、日志时间和通知时间等高频访问方向，因此能够满足当前系统的数据访问需求。

2.4 安全认证与可视化技术

系统采用 Spring Security 与 JWT 实现无状态认证。用户登录后获得 access token 和 refresh token，前端在后续请求中通过 Authorization 头传递令牌；后端解析令牌并将当前用户、角色和权限信息放入安全上下文。该方案能够较好匹配前后端分离项目的认证需求。

安全设计不仅覆盖登录认证，还包括 refresh token 持久化、刷新令牌轮换、登录日志、操作日志、安全响应头和部分高风险接口限流。ECharts 则为库存占比、趋势统计、仓库分布和效期分析提供图形化展示能力，使管理人员能够更直观地查看库存与业务变化情况。

2.5 本章小结

本章结合项目实际使用的技术栈，说明了前端展示、后端服务、数据库持久化、认证授权与数据可视化等关键技术的选型依据。上述技术共同构成了系统后续需求实现、业务建模和测试验证的技术基础。

3 系统需求分析

3.1 可行性分析

3.1.1 技术可行性

系统采用 Spring Boot、Vue 3、MySQL、JWT 和 MyBatis-Plus 等成熟技术栈，相关框架在当前项目中已完成真实落地，后端测试、前端构建、单元测试和端到端测试也已具备执行基础，因此从技术实现角度具备较强可行性。现有仓库已经包含核心业务代码、数据库脚本和测试材料，可以直接支撑论文中的设计说明与验证。

3.1.2 经济可行性

本系统主要采用开源框架和常见开发工具，开发环境仅需要普通教学或实验用计算机即可完成编码、调试和测试。部署形态以单机或校内局域网环境为主，不依赖高成本硬件和商业中间件，因此建设与维护成本相对可控。

3.1.3 社会可行性

高校物资管理涉及教学保障、后勤服务和应急支撑，信息化建设有助于减少纸质台账和重复沟通，提高库存追踪、审批留痕和风险提醒的及时性。系统以规范流转和过程可追溯为目标，符合校园管理数字化、精细化的发展方向。

3.1.4 法律可行性

系统面向校内物资管理场景，核心数据为用户、库存、单据、日志和通知等业务信息，不涉及超出课题范围的敏感社会数据。通过角色权限控制、登录认证、日志审计和接口边界约束，系统能够在现有校园管理制度下满足基本的合规使用要求。

3.2 功能性需求分析

3.2.1 系统结构分析

系统面向高校物资管理场景，核心用户包括系统管理员、仓库管理员、审批人员和部门用户。系统管理员维护用户、角色、部门、校区、物资分类、物资档案、供应商和系统配置；仓库管理员负责仓库、库位、库存、入库、出库、调拨执行和预警处理；审批人员负责申领单和调拨单审核；部门用户主要完成物资申领、查看流转状态和签收。该划分与当前项目的菜单、接口和种子数据保持一致。

表 3-1 系统功能需求概括

功能域	需求说明
基础数据管理	维护用户、角色、部门、校区、仓库、库位、物资分类、物资档案、供应商和系统配置，为后续业务单据提供基础引用。
库存与批次管理	支持库存分页查询、入库建批次、批次余量查询、出库扣减和库存盘点。库存汇总与批次明细分表管理。
申领审批与出库	部门用户创建申领单，提交后锁定库存；审批通过后仓库管理员按申领单执行出库，部门用户完成签收。
调拨协同	仓库管理员创建调拨单，审批人员审核后执行跨仓批次搬移，目标仓生成镜像批次并更新库存汇总。
预警统计与支撑	支持低库存、积压、临期、过期、异常出库预警；支持补货建议、统计图表、通知消息、登录日志和操作日志。

3.2.2 基础数据管理需求

基础数据是系统运行的前提。物资档案中的物资编码、名称、分类、规格、单位、安全库存、保质期和供应商信息会被库存、预警、申领和补货建议反复引用；仓库和库位数据决定库存记录和调拨单的归属；部门和角色决定用户可以访问的菜单与业务流程。

3.2.3 库存批次管理需求

校园物资管理不能只依赖一个总库存数字。系统需要同时保存某物资在某仓库中的汇总库存和每个批次的余量、生产日期、到期日期。这样才能在出库时优先使用更早到期的批次，并在临期或过期时生成预警。

3.2.4 申领审批需求

申领流程要求部门用户先创建单据，再提交进入审批。提交时系统根据可用库存选择满足物资需求的仓库并锁定库存，避免多个申请同时提交造成库存超占。审批通过后单据进入可出库状态，出库完成后部门用户签收，形成业务闭环。

3.2.5 调拨协同需求

调拨流程用于处理跨仓库存不平衡。系统需要保存调出仓、调入仓、调拨物资和数量，并通过提交、审批、执行、签收等状态节点记录协同过程。推荐功能只根据固定校区图和库存余量对候选仓排序，不承担完整物流调度。

3.2.6 预警统计需求

预警需求包括低库存、库存积压、临期、过期和异常出库。统计需求包括运营总览、库存占比、出入库趋势、仓库分布、部门排行、效期统计等。相关结果用于辅助管理人员发现库存风险和了解物资流转情况。

3.2.7 关键业务流程与用例说明

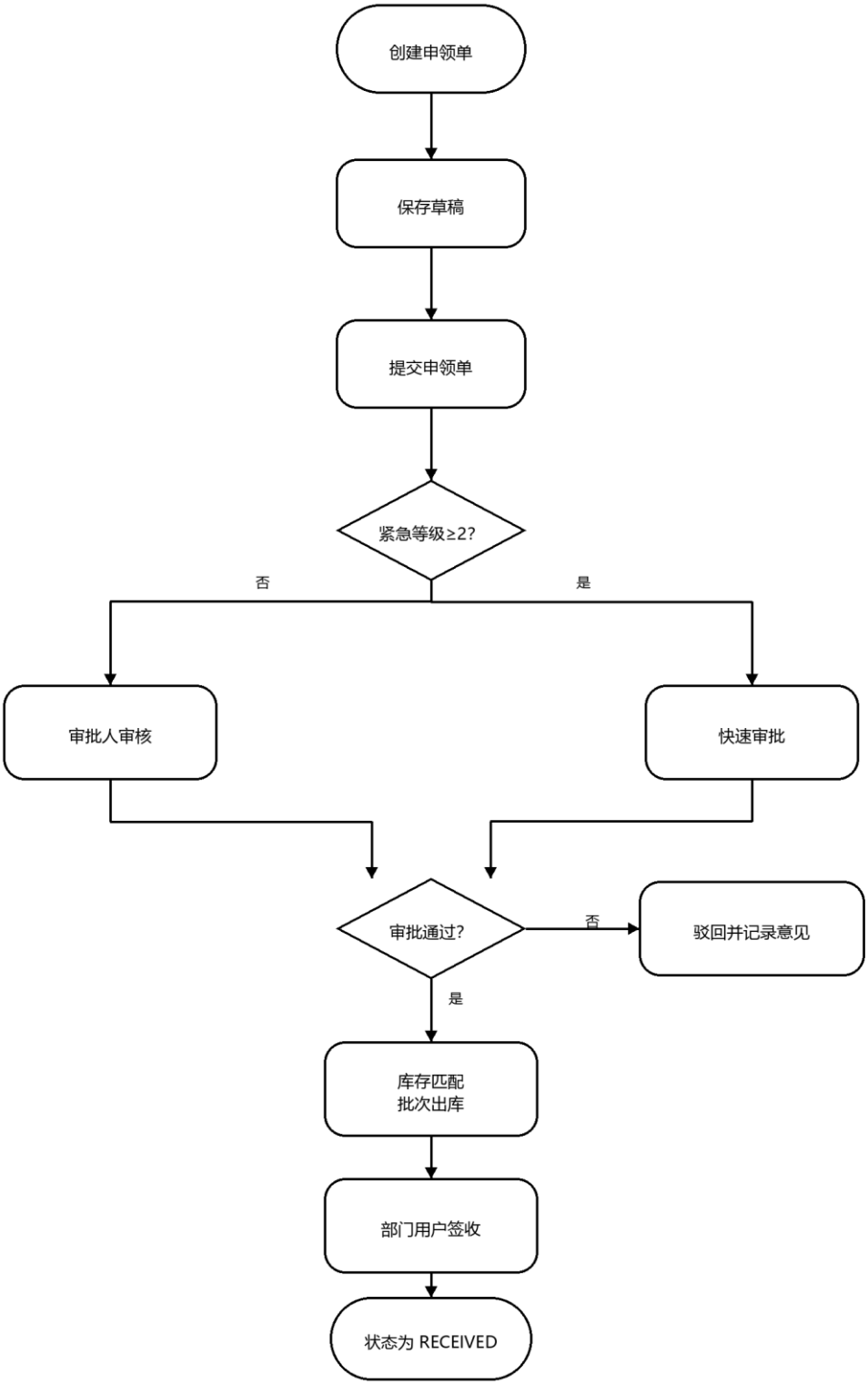

图 3-1 申领审批闭环流程图

申领审批流程从部门用户创建申领单开始，经过提交、审批、出库和签收。系统在提交阶段锁定库存，在出库阶段回写实发数量，并在签收阶段释放未使用的锁定量。

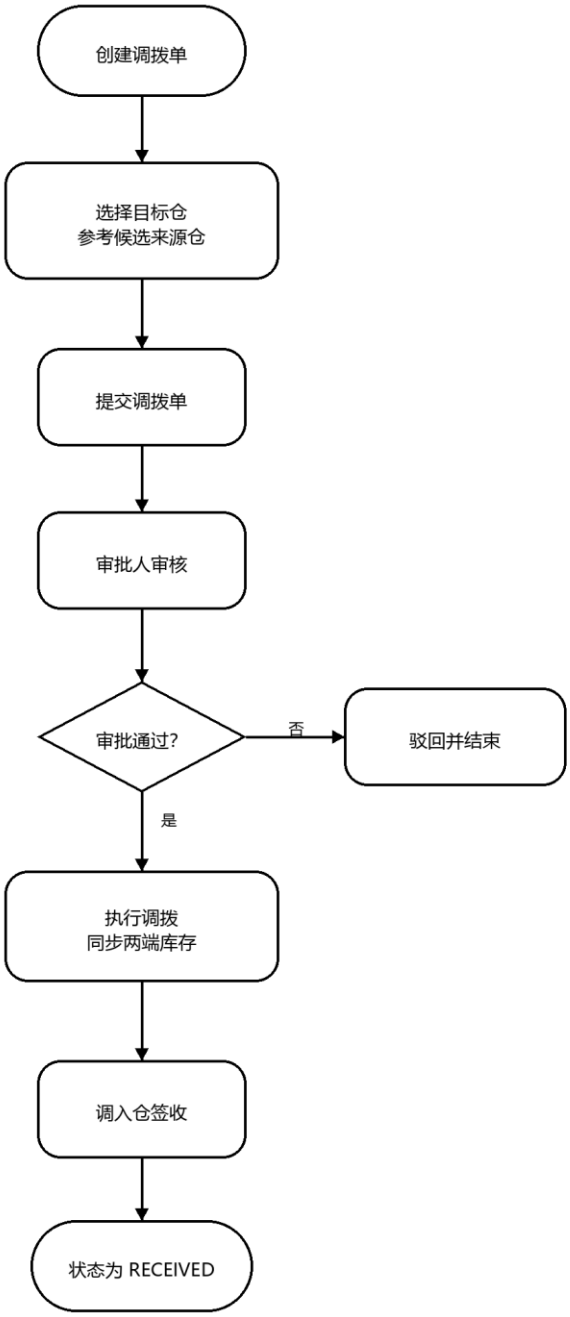

图 3-2 调拨执行流程图

调拨执行流程强调跨仓协同。调拨单审批通过后，服务层按 FEFO 顺序扣减来源仓批次，并为目标仓生成包含调拨单号的新批次记录。

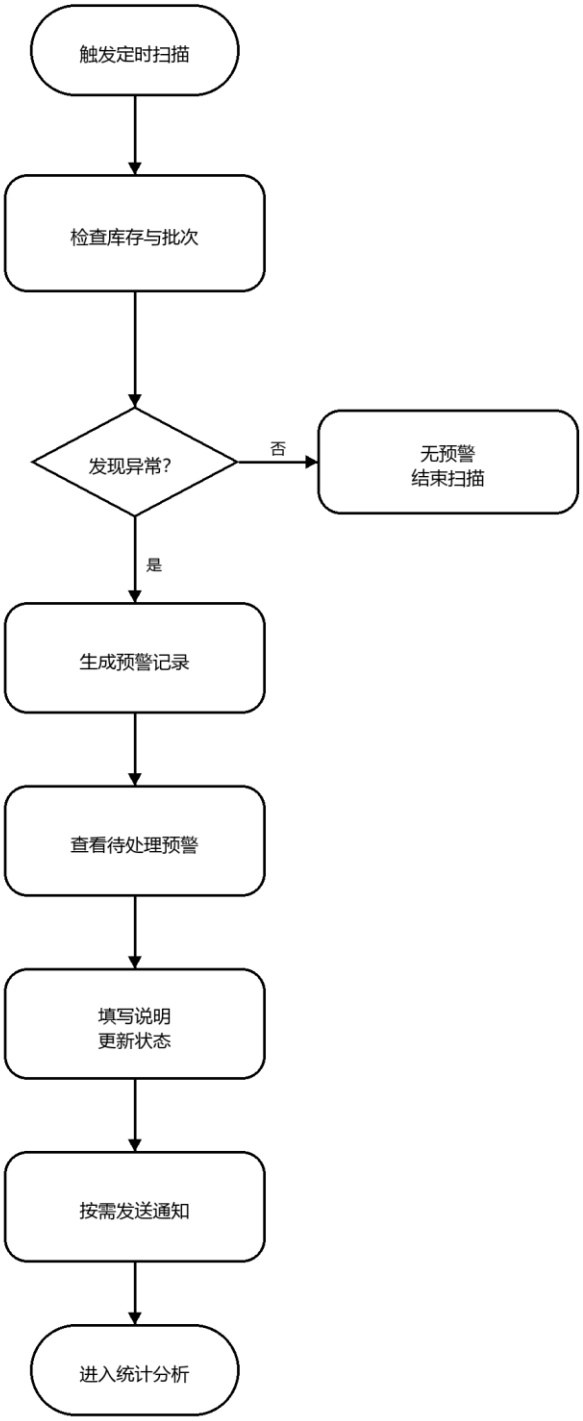

图 3-3 预警处置流程图

预警处置流程包括定时或手动扫描、预警去重、记录生成和人工处理。系统只对未处理预警做去重，避免同一风险重复堆积。

3.3 非功能性需求分析

除功能性需求外，系统还需要满足可用性、安全性、可维护性和可扩展性等非功能要求。

前端页面应保证常见业务操作路径清晰，后端接口应提供统一返回结构和分页契约，认证与日志机制应支持权限控制和过程追踪。系统在当前实现范围内的主要非功能要求如表 3-2 所示。

表 3-2 系统非功能需求

类别	要求
安全性	登录认证、JWT 校验、角色菜单、接口授权、安全响应头、登录日志和操作日志。
一致性	库存汇总、批次余量、申领实发数量和单据状态在同一事务内更新。
可维护性	后端按模块拆分 Controller、Service、Mapper、Entity，前端按视图和通用组件组织。
可测试性	提供后端单元与集成测试、前端单测、Playwright E2E 和 k6 脚本。
可观测性	提供 Actuator、Prometheus 指标、JSON 日志和基础观测配置。

3.4 本章小结

本章从可行性、功能性和非功能性三个层面分析了系统需求，明确了角色边界、核心业务链路与系统应满足的质量要求，并结合关键业务流程对申领、调拨和预警处置进行了说明。这些分析为后续系统设计与功能实现提供了直接依据。

4 系统设计

4.1 系统总体架构设计

系统采用 B/S 架构和前后端分离设计。用户通过浏览器访问 Vue 前端，前端通过 Axios 调用 Spring Boot API；后端负责认证、权限、业务规则和数据访问；MySQL 保存正式业务数据，H2 用于测试和截图环境。该设计与当前仓库 `docs/architecture.md`、`backend/pom.xml` 和 `frontend/package.json` 保持一致。

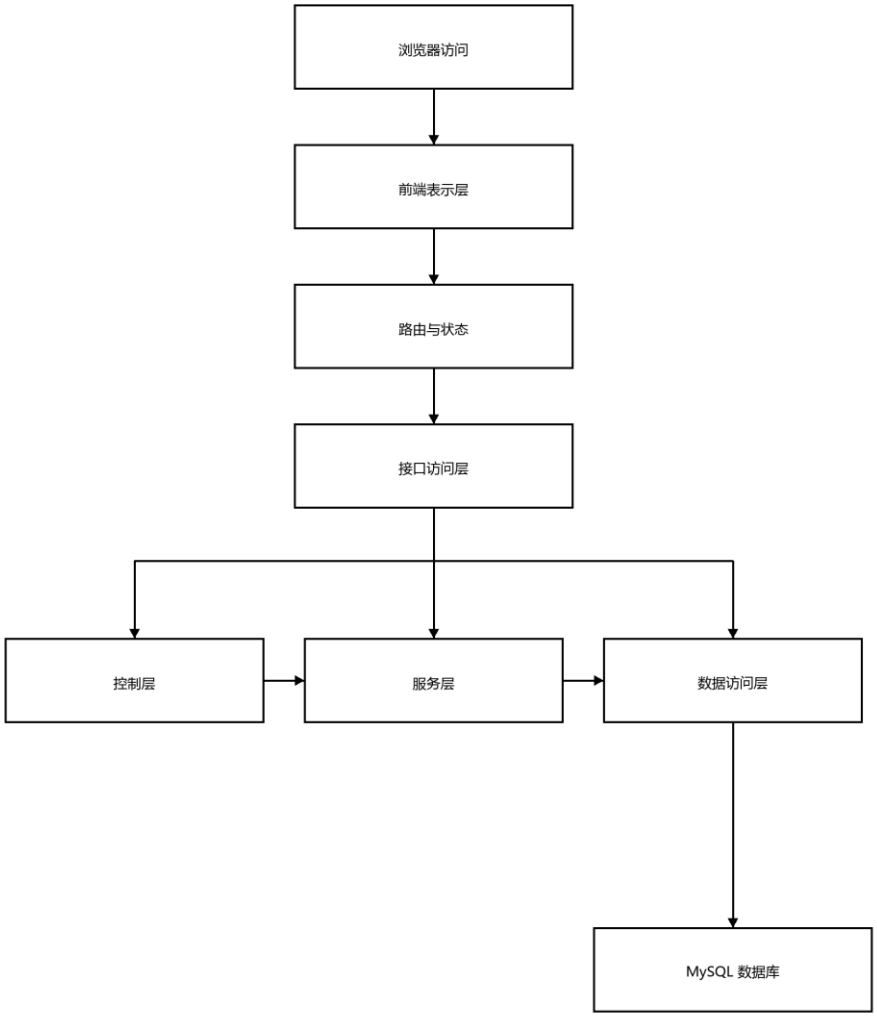

图 4-1 系统总体架构图

4.2 系统模块设计

系统功能模块按照实际代码目录和前端路由划分，包括认证授权、RBAC、基础数据、物资档案、仓库库位、库存出入库、申领审批、调拨管理、预警中心、统计分析、通知消息、事件管理和日志审计。模块之间通过业务单据和数据库主键关联，不采用微服务拆分。

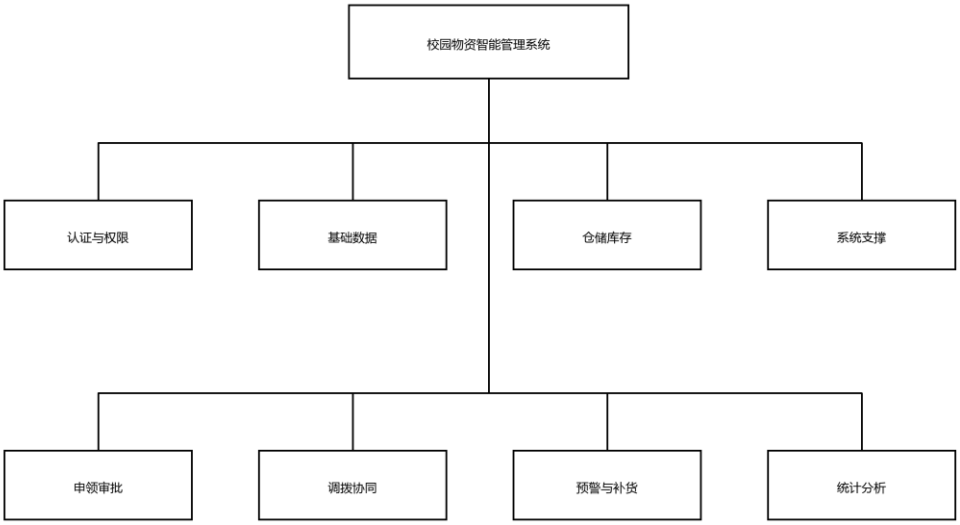

图 4-2 系统功能模块图

4.2.1 接口与安全设计

系统接口统一以 `/api` 为前缀，登录、刷新、当前用户和菜单由 `AuthController` 提供；申领、库存、调拨、预警、统计、通知和日志分别由对应 Controller 提供。接口响应采用 `ApiResponse` 统一包装，分页接口使用 `PageQuery` 和 `PageResult`，返回 records、total、page 和 size。

安全设计包括三层边界：第一层是登录认证，用户通过用户名和 BCrypt 密码校验获取令牌；第二层是 JWT 过滤器，后端从 Authorization 头解析用户身份；第三层是角色与方法级权限，控制不同角色可访问的菜单和接口。refresh token 以 hash 形式持久化，刷新成功后撤销旧令牌并签发新令牌。

4.2.2 智能能力设计

本文所述智能能力只指规则化和统计化辅助能力。入厂煤智能平台和智慧实验室研究中存在流程自动化、预警模型和人工智能应用[21][22]，但本系统没有实现复杂预测、图像识别或深度学习模型。系统当前可写的智能能力如表 4-1 所示。

表 4-1 智能能力与实现方式对照表

能力	实现依据	写作边界
库存预警	WarningService 扫描 currentQty 与 safetyStock	写低库存、积压等规则，不写预测模型。
效期提醒	inventory_batch.expire_date 与当前日期比较	写临期、过期提醒。
FEFO 出库	InventoryService 按 expire_date、id 排序扣减批次	写批次优先规则。
调拨推荐	DijkstraUtil 固定校区图 + TransferService 库存筛选	写候选仓排序，不写物流调度。
补货建议	SmartService 根据 30 天出库量和保障天数计算建议量	写补货参考。
移动平均预测	SmartService 使用近 6 个月出库历史均值	写参考预测，不写模型训练。
统计分析	AnalyticsController 聚合库存和出入库数据	写可视化统计。

4.3 数据库设计

4.3.1 数据库设计原则

数据库设计以 `sql/schema.sql` 为唯一依据。系统将用户角色、物资主数据、库存汇总、库存批次、业务单据、预警记录、通知消息和日志审计分别建表，既避免把所有业务混在一张表中，也避免设计当前代码不使用的冗余表。

4.3.2 实体关系设计

系统实体关系可分为三组。第一组是用户、角色和部门，用于支撑身份认证和菜单权限；第二组是物资、仓库、库存和批次，用于支撑物资存储与出入库；第三组是申领单、调拨单、预警、通知和日志，用于支撑业务闭环与审计。

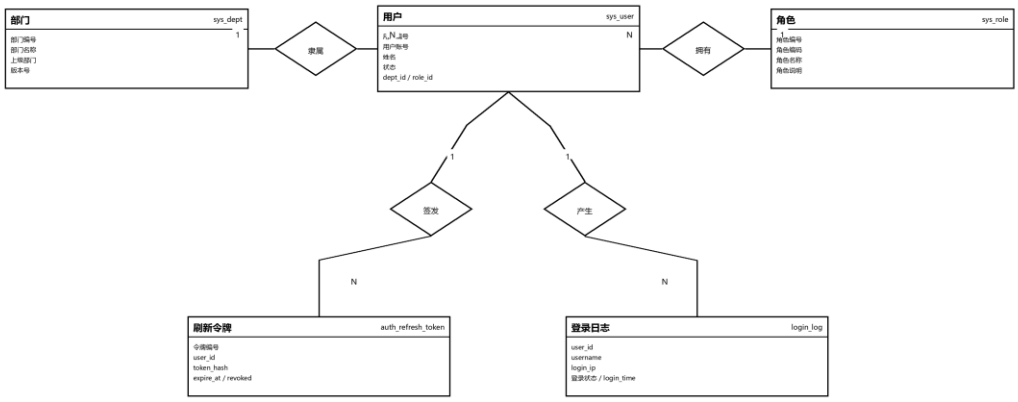

图 4-4 用户角色与组织 E-R 图

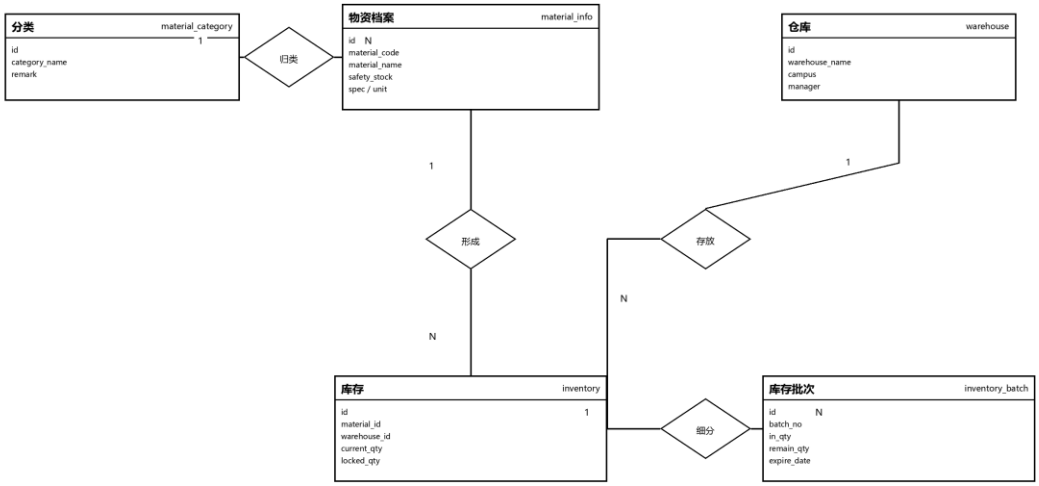

图 4-5 库存与批次 E-R 图

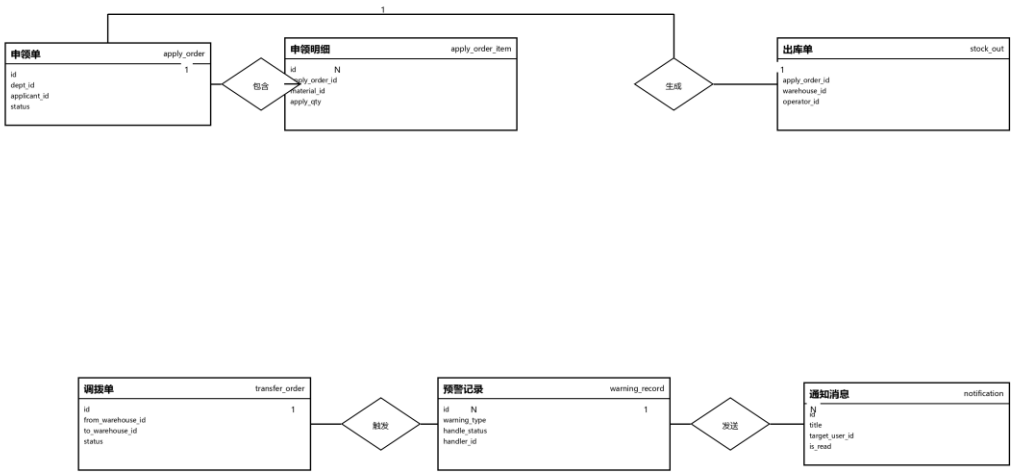

图 4-6 业务单据、预警与通知 E-R 图

4.3.3 关键数据表设计

表 4-2 关键数据表说明

数据表	主要字段	设计作用
sys_user	username、password、real_name、dept_id、	系统账号和角色归属。

	role_id、status	
sys_role	role_code、role_name、description	角色编码与名称。
sys_dept	dept_name、parent_id	组织部门层级。
auth_refresh_token	user_id、token_id、token_hash、expire_at、revoked	刷新令牌持久化和撤销。
material_info	material_code、material_name、category_id、safety_stock、shelf_life_days	物资档案与安全库存。
warehouse	warehouse_name、campus、address、manager	仓库基础信息。
inventory	material_id、warehouse_id、current_qty、locked_qty	物资仓库维度库存汇总。
inventory_batch	material_id、warehouse_id、batch_no、remain_qty、expire_date	批次库存与效期。
apply_order	dept_id、applicant_id、urgency_level、status、reserved_warehouse_id	申领主单。
apply_order_item	apply_order_id、material_id、apply_qty、actual_qty	申领明细。
stock_in / stock_out	warehouse_id、operator_id、apply_order_id、remark	入库和出库主单。
transfer_order	from_warehouse_id、to_warehouse_id、status、approver_id	调拨主单。
warning_record	warning_type、material_id、warehouse_id、handle_status	预警记录。
notification	title、content、msg_type、target_user_id、is_read	站内通知。
login_log / operation_log	用户、模块、操作、时间、状态	登录与操作审计。

4.3.4 索引与约束设计

系统索引围绕实际查询和状态过滤设计。`sys_user.username`、`sys_role.role_code`、`material_info.material_code`和`inventory(material_id, warehouse_id)`使用唯一约束保证关键业务对象不重复。批次出库使用`idx_batch_outbound_pick(material_id, warehouse_id, expire_date, remain_qty)`支撑按物资、仓库和效期查找候选批次；预警查询使用`idx_warning_type_material_warehouse_status`支撑去重和状态筛选；日志和通知

按时间与用户建立索引，便于分页查询。

4.4 本章小结

本章完成了系统总体架构、功能模块、接口与安全边界、规则化智能能力以及数据库结构的设计说明，明确了前后端协作方式、核心业务对象及其数据关系。上述设计结果为后续系统实现与测试章节中的代码说明和运行验证提供了结构基础。

5 系统实现

5.1 开发环境搭建

表 5-1 系统开发与运行环境

类别	内容
后端	Java 17，Spring Boot 3.3.5，Spring Security，MyBatis-Plus 3.5.7，JJWT，Bucket4j，Caffeine。
前端	Vue 3.5.13，Vite 6.2.0/6.4.1，Pinia 2.3.1，Element Plus 2.9.7，ECharts 5.6.0，Axios。
数据库	MySQL 8 用于业务数据，H2 用于测试、截图和 E2E 隔离环境。
测试	JUnit、Spring Boot Test、Vitest、Playwright、k6。
运行接口	后端 8080，管理端点 18080，前端开发/预览端口 5173 或 4173。

5.2 功能实现

5.2.1 认证授权与登录态管理实现

认证授权由 `AuthController`、`AuthService`、`JwtTokenProvider`、`JwtAuthenticationFilter` 和 `SecurityConfig` 共同完成。登录时系统校验用户名、账号状态和 BCrypt 密码，成功后签发 access token 和 refresh token，并记录登录日志。refresh token 不以明文保存，而是保存 token_id、token_hash、过期时间和撤销状态。

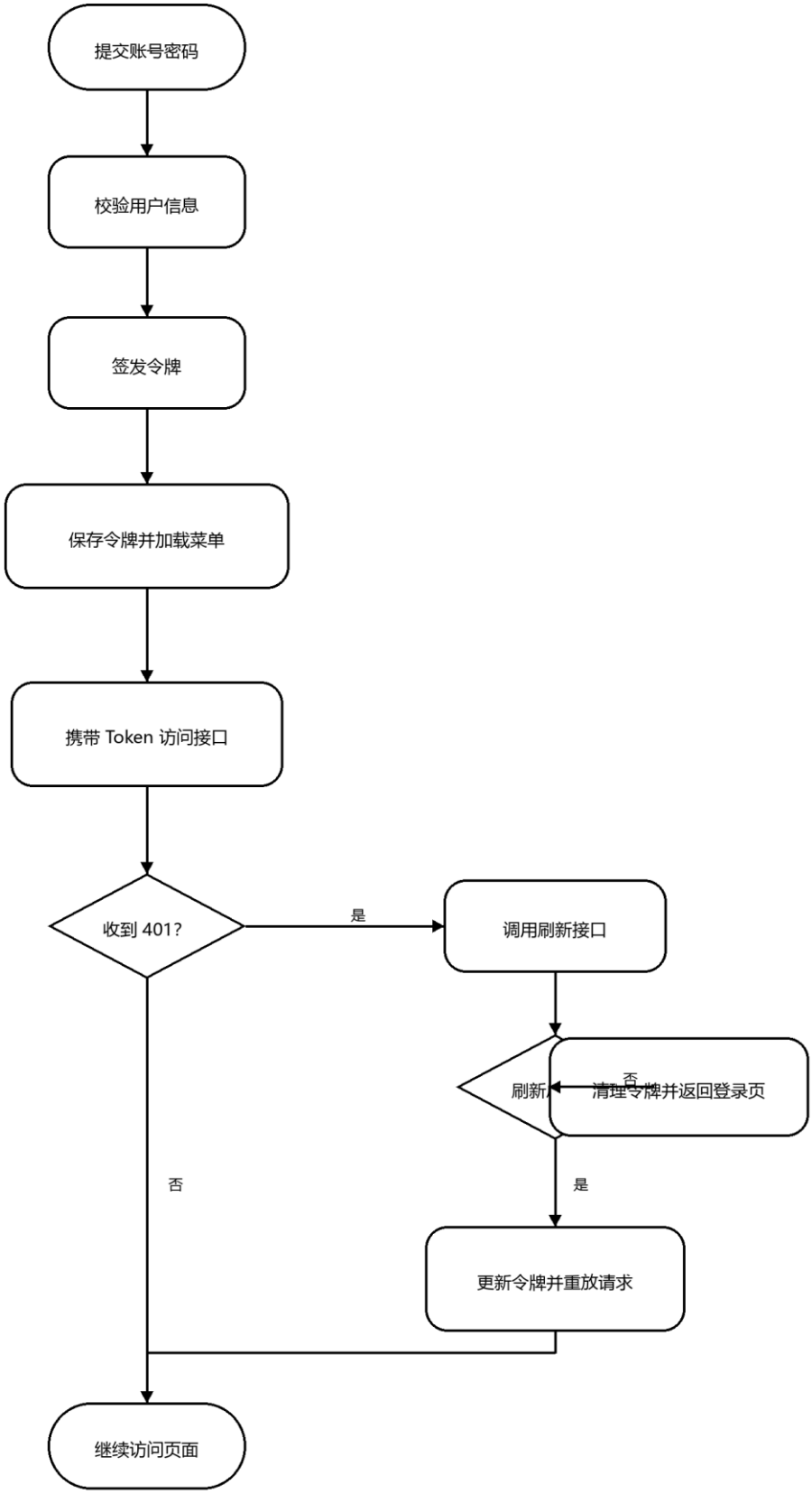
提交账号密码
校验用户信息
签发令牌
保存令牌并加载菜单
携带 Token 访问接口
收到 401?
是
调用刷新接口
刷新
否
清理令牌并返回登录页
是
更新令牌并重放请求
否
继续访问页面

图 5-1 登录认证与令牌续签流程图

图 5-2 系统登录界面

刷新令牌采用单次使用策略。服务端校验 refresh token 的类型、jti、hash 和过期时间，校验通过后先撤销旧记录，再生成新的 token 对。该设计能够减少刷新令牌重复使用风险。

5.2.2 基础数据管理实现

基础数据模块包括用户、角色、部门、校区、仓库、库位、物资分类、物资档案、供应商和系统配置。后端相关模块位于 `rbac`、`campus`、`warehouse`、`material`、`supplier` 和 `config` 目录，前端对应 `UserView`、`DeptView`、`CampusView`、`WarehouseView`、`MaterialView` 等页面。保存操作通过 MyBatis-Plus 完成实体持久化，删除操作采用逻辑删除字段。

5.2.3 库存批次与出入库实现

库存模块由 `InventoryService` 实现。入库时系统先创建 `stock_in` 主单和 `stock_in_item` 明细，再增加 `inventory` 汇总库存，并创建 `inventory_batch` 批次记录。出库时系统先校验汇总库存和锁定量，再按 `expire_date ASC, id ASC` 查询候选批次并扣减余量，最后更新汇总库存、出库明细和预警状态。

图 5-3 库存查询界面

申领出库与普通出库的差别在于，申领出库需要检查申领单是否已锁定到当前仓库，并在出库后回写 `apply_order_item.actual_qty`。这些操作位于同一事务中，避免库存汇总、批次余量和申领单明细不一致。

5.2.4 申领审批与调拨实现

申领服务由 `ApplyService` 实现。部门用户创建申领单时生成 DRAFT 状态；提交时系统聚合申领明细，选择可满足全部物资需求的仓库并锁定库存。普通申领进入 SUBMITTED 状态等待审批，紧急等级达到阈值时走快速通道并记录自动审批信息。审批通过后进入 APPROVED 状态，仓库管理员执行出库后进入 OUTBOUND，部门用户签收后进入 RECEIVED。

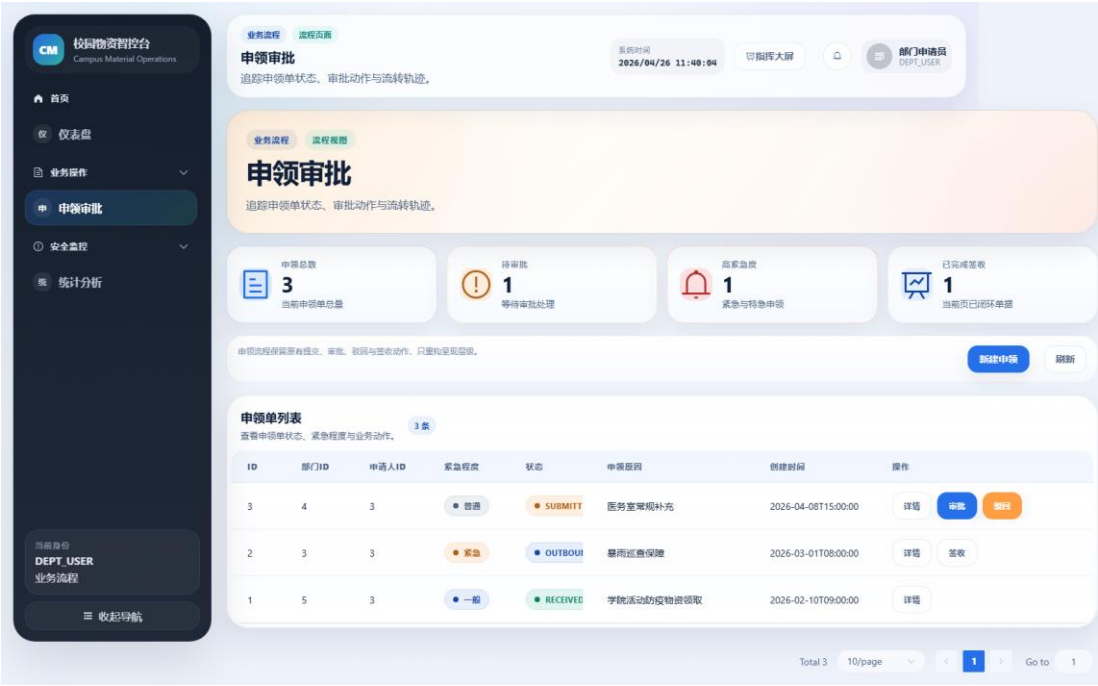

图 5-4 部门用户申领界面

调拨服务由 `TransferService` 实现。调拨单创建后处于 DRAFT 状态，提交后由审批人员审核。执行调拨时，系统按 FEFO 顺序扣减来源仓批次，并在目标仓生成带有调拨单号后缀的镜像批次，随后分别更新来源仓和目标仓的库存汇总。

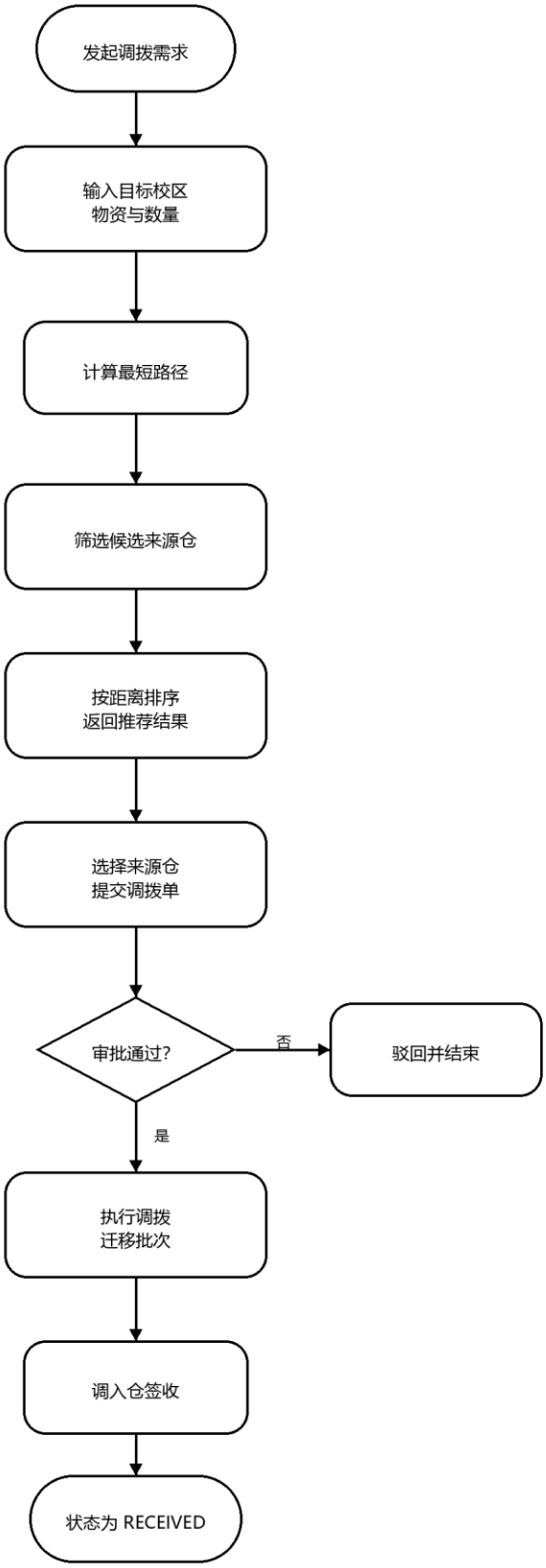

图 5-5 调拨执行与候选仓排序流程图

图 5-6 调拨管理界面

5.2.5 预警、补货建议与统计分析实现

预警扫描由 `WarningService.scan` 实现，包含低库存、积压、临期、过期和异常出库五类规则。低库存规则判断当前库存是否低于安全库存；积压规则判断当前库存是否超过安全库存 3 倍；临期规则查找 30 天内到期且仍有余量的批次；过期规则查找已超过到期日的批次；异常出库规则比较近 7 天出库量与近 30 天折算周均出库量。

图 5-7 预警管理界面

补货建议和移动平均预测由 SmartService 实现。补货建议根据物资安全库存、当前库存、近 30 天出库量和保障天数计算目标库存与建议补货量；预测接口按近 6 个月出库历史聚合月度数量，并用平均值生成未来月份参考值。该功能用于生成补货参考和基于移动平均的趋势预测，不涉及模型训练。

图 5-8 统计分析界面

5.2.6 日志通知与事件管理实现

日志模块包括登录日志和操作日志。登录日志记录用户、用户名、登录 IP、登录状态、登录时间和 user-agent；操作日志记录操作者、模块、操作类型和详情。通知模块提供未读数量、标记已读、全部已读和删除功能。事件管理模块用于记录校园突发事

件的标题、类型、级别、校区、地点、描述、处理人和状态。

5.3 本章小结

本章围绕开发环境、认证授权、基础数据、库存批次、申领审批、调拨、预警统计以及日志通知等功能，说明了系统的主要实现过程，并结合流程图与运行截图展示了关键业务链路。实现部分始终以真实代码与接口行为为依据，避免脱离项目去罗列无关代码片段。

6 系统测试

6.1 系统安装部署

系统采用前后端分离方式部署。后端在 screenshot 或 dev profile 下通过 Spring Boot 启动，默认提供 8080 业务接口和 18080 管理端口；前端通过 npm 安装依赖后使用 Vite 启动或构建静态资源。正式运行时可连接 MySQL，截图与端到端验证则使用隔离的 H2 数据集，以便在不影响正式数据的情况下完成测试与演示。
测试阶段的主要环境包括 Java 17、Spring Boot 3.3.5、Node.js、Vue 3、MySQL 8、H2、Maven、Vitest、Playwright 和 k6。安装部署的重点不是堆砌环境说明，而是保证后端服务、前端页面、数据库脚本和自动化测试工具能够协同运行，为后续功能测试提供稳定基础。

6.2 软件功能测试

6.2.1 后端自动化测试

2026 年 4 月 26 日重新执行 `mvn test`，后端共 49 项测试通过，失败 0 项、错误 0 项、跳过 0 项。测试范围包括统一响应、业务异常、全局异常处理、登录刷新、限流、申领出库主链路、并发申领库存锁定、调拨执行、预警处理和分页接口。

表 6-1 后端自动化测试执行情况

测试对象	覆盖内容	结果
common	ApiResponse、BizException、GlobalExceptionHandler	通过
auth	登录、refresh token 轮换、令牌清理、安全响应头	通过
apply	申领创建、提交、审批、驳回、签收	通过
inventory	申领出库、库存锁定、批次扣减	通过
transfer	调拨创建、审批、执行和库存搬移	通过
warning	预警处理和高风险接口限流	通过
integration	核心流程与分页响应契约	通过

6.2.2 前端构建、单测与 E2E 验证

前端执行 `npm run build` 后,Vite 生产构建成功;执行 `npm run test:unit` 后，3 个测试文件、8 项测试通过；执行 `npm run test:e2e` 后，4 个端到端场景通过。E2E 场景包括管理员登录查看库存页面、部门用户创建申领单、仓库管理员创建调拨单、

仓库管理员处理预警。

表 6-2 前端验证结果

命令	验证内容	结果
npm run build	生产构建和资源打包	通过
npm run test:unit	HTTP 封装、登录页、状态标签组件	3 个文件 8 项通过
npm run test:e2e	登录、库存、申领、调拨、预警流程	4 个场景通过

6.2.3 业务场景测试

业务场景验证主要围绕系统核心流程。申领审批场景验证从部门用户创建申领单到提交、审批、出库和签收的状态变化；调拨场景验证来源仓扣减、目标仓增加和批次镜像生成；预警场景验证手动扫描和处理动作；认证场景验证登录、刷新、旧 refresh token 失效和安全响应头。

表 6-3 典型业务场景验证说明

场景	验证点	证据
登录认证	access token、refresh token、安全响应头、登录日志	CoreFlowIntegrationTest、AuthServiceTest
申领审批	DRAFT、SUBMITTED、APPROVED、OUTBOUND、RECEIVED 状态链路	CoreFlowIntegrationTest、ApplyServiceTest
库存出库	锁定量校验、FEFO 批次扣减、实发数量回写	InventoryService、集成测试
调拨执行	来源仓扣减、目标仓增加、镜像批次生成	TransferServiceTest、E2E
预警处理	未处理预警处理为已处理，高风险接口限流	WarningControllerTest、E2E

6.2.4 性能基线与边界说明

仓库中提供了 tests/performance 下的 k6 脚本和 docs/performance-baseline.md 基线记录。该基线记录采集于本地 screenshot profile 和隔离 H2 数据集，场景包括登录、库存分页列表、预警分页列表和操作日志分页列表，适用于相同环境下的基线对比，不用于推导生产服务承诺或大规模并发结论。

表 6-4 本地 k6 基线记录

场景	脚本	错误率	p95
登录	tests/performance/login.js	0.00%	78.20 ms
库存分页列表	tests/performance/inventory-list.js	0.00%	96.51 ms
预警分页列表	tests/performance/warning-list.js	0.00%	20.42 ms

操作日志分页列表	tests/performance/operation-log-list.js	0.00%	17.41 ms

6.3 本章小结

本章从安装部署、自动化测试、前端验证、业务场景测试和性能基线等方面对系统进行了验证。测试结果表明，系统在当前项目范围内能够较稳定地支撑登录认证、库存管理、申领审批、调拨执行和预警处理等核心功能，同时也明确了性能数据仅适用于本地基线对比的使用边界。

结束语

本文围绕校园物资智能管理系统完成了需求分析、总体设计、数据库设计、详细实现和系统测试。系统采用前后端分离架构，围绕校园物资管理中的基础数据、库存批次、申领审批、调拨协同、预警处理、通知消息、日志审计和统计分析建立了较完整的业务流程。

从实现结果看，系统已经能够支持多角色登录和菜单边界控制，并完成库存入库、批次维护、按效期优先出库、申领审批闭环、跨仓调拨、库存风险预警和补货参考等核心功能。测试验证表明，后端自动化测试、前端构建、前端单测和端到端流程均已通过。

系统仍存在后续完善空间。第一，可以继续细化采购计划、供应商协同和报废处置流程；第二，可以在更充分的数据积累后尝试更精细的需求预测和异常检测；第三，可以扩展移动端或扫码核验能力；第四，可以在真实部署环境中继续完善数据库备份、权限审计和运行监控。上述内容属于未来工作，本文不将其作为已经实现的系统功能。

致　谢

在毕业设计过程中，指导教师在选题确定、系统实现、结构组织和格式规范方面给予了耐心指导。软件学院提供的课程训练和毕业设计要求，使我能够将软件工程方法、数据库设计、前后端开发和测试验证结合起来完成本课题。

同时感谢同学和家人在资料整理与系统演示过程中的支持。通过本次毕业设计，我进一步认识到工程项目应以真实需求和可验证证据为基础，并在实现过程中保持规范和严谨。

参考文献

[1] 李晓英，高菲. 智慧校园背景下的高职院校物资管理系统建设创新研究[J]. 现代物业(中旬刊)，2019(09)：12.

[2] 唐锡雷. 基于高职院校的后勤物资管理系统设计与实现[D]. 成都：电子科技大学，2015.

[3] 曾文勇，刘小玲. 数智赋能物资供应管理提质提效[N]. 中国石化报，2026-04-24(002).

[4] 张俊萌，张华贵. 医疗机构物资流通管理信息系统的设计与实现[J]. 工业控制计算机，2026，39(04)：132-134.

[5] 厉志安，李笑，葛江勤，等. 基于物联网的高校实验室危化品溯源智能管理[J]. 实验室研究与探索，2025，44(03)：258-263.

[6] 崔靖茹，文华，刘宏磊，等. 基于Vue和SpringBoot框架的高校信息化项目管理系统的设计与实现[J]. 现代信息科技，2025，9(22)：77-81.

[7] 薛楠楠，张伟，钟化雨，等. 基于 SpringBoot+Vue 的事故报告数据库系统设计与实现[J]. 计算机仿真，2025，42(07)：456-462+570.

[8] 李琴，崔名扬，钱奕文，等. 基于 SpringBoot 的研究生学术档案管理系统开发[J]. 电脑知识与技术，2023，19(18)：46-48+51.

[9] 沈露，孙雨晨，义智文宇，等. 基于互联网的爱心校园服务平台设计与实现[J]. 电脑编程技巧与维护，2023(06)：56-58.

[10] Kevin Santiago Rey Rodriguez, Julián David Avellaneda Galindo, Josep Tárrega Juan, et al. Secure Development Methodology for Full Stack Web Applications: Proof of the Methodology Applied to Vue.js, Spring Boot and MySQL[J]. Computers, Materials & Continua, 2025, 85(1).

[11] Yuanrun Zhu. Contract Management System Based on SpringBoot and Vue[J]. Advances in Computer, Signals and Systems, 2024, 8(5).

[12] Yixuan Liu. Design and Implementation of a Student Attendance Management System based on Springboot and Vue Technology[J]. Frontiers in Computing and Intelligent Systems, 2024, 8(1).

[13] Jian Chen, Chen Jian, Pan Hailan. Design of Man Hour Management Information System on SpringBoot Framework[J]. Journal of Physics: Conference Series, 2020, 1646(1).

[14] 王晴. 基于 Java 和 MySQL 技术的供求信息网设计与实现[J]. 兰州石化职业技术大学学报，

2025, 25(04): 26-31.

[15] 王阳坤, 陈宇峰, 向郑涛, 等. 基于 Vue 框架的动态路由监控系统设计[J]. 湖北汽车工业学院学报, 2026, 40(01): 8-11.

[16] 陆岫昶, 林娜, 骆永翰, 等. Vue 支持下的多层次协同可视化前端设计[J]. 电子设计工程, 2025, 33(19): 127-131.

[17] 张文亮. MySQL 应用实战与性能调优[M]. 北京: 机械工业出版社, 2022.

[18] 袁子伯, 杨富源. 基于 MySQL 的数据库查询性能优化技术研究[N]. 农业科技报, 2026-02-02(006).

[19] 卢万有. 基于 JWT 的 RBAC 在前后端分离项目中的设计与实现[J]. 电脑编程技巧与维护, 2025(01): 46-48.

[20] 胡传甫, 王昕葳, 周宬, 等. 业务逻辑漏洞常见类型分析与 JWT 测试示例研究[J]. 计算机时代, 2025(02): 6-10.

[21] 张迪, 刘小翔, 姚欢欢, 等. 基于 B/S 架构的入厂煤智能管理平台设计与开发[J]. 自动化应用, 2026, 67(07): 222-224+228.

[22] 杨职远, 马海龙. 基于人工智能的智慧实验室管理系统的设计与应用[J]. 软件, 2025, 46(11): 28-30.

附录 1 关键接口与测试补充说明

附录内容用于补充正文中不宜展开的接口入口与测试范围说明，所有条目均来源于当前项目的控制器、服务类和测试文件。

附表 1-1 关键业务接口补充说明

功能域	主要接口	对应模块	说明
认证授权	/api/auth/login、/api/auth/refresh、/api/auth/me、/api/auth/menus	AuthController、AuthService	完成登录、续签、当前用户和角色菜单加载。
库存与批次	/api/inventory/list、/api/inventory/batches、/api/inventory/stock-in、/api/inventory/stock-out	InventoryController、InventoryService	支撑库存查询、批次查询、入库和出库。
申领审批	/api/apply/list、/api/apply/{id}/submit、/approve、/reject、/receive	ApplyController、ApplyService	支撑申领单提交、审批、驳回和签收。
调拨管理	/api/transfer/list、/api/transfer/recommend、/submit、/approve、/execute、/receive	TransferController、TransferService	支撑调拨推荐、审批、执行和签收。
预警与统计	/api/warning/list、/api/warning/scan、/api/smart/*、/api/analytics/*	WarningService、SmartService、AnalyticsService	支撑预警扫描、补货参考、移动平均预测和图表统计。
日志通知	/api/login-log/*、/api/log/list、/api/notification/*	Log、Notification 模块	支撑登录日志、操作日志和通知消息。

附录测试范围包括后端 Maven 自动化测试、前端 Vitest 单元测试、Vite 生产构建、Playwright E2E 和 k6 脚本。对应命令为 `mvn test`、`npm run test:unit`、`npm run build`、`npm run test:e2e` 和 `k6 run tests/performance/*.js`。